

Bilan d'une SAÉ — Fiches portfolio

Rayan SAOUD · BUT R&T · SAÉ 4.DevCloud.01 — Gestion bancaire en microservices

FICHE 1 · AVANT — FICHE DE SUIVI

Fiche de suivi de la SAÉ

Préparée au démarrage du projet pour cadrer le travail

Nom de la SAÉ	SAÉ 4.DevCloud.01 — Application de gestion bancaire en microservices
Objectifs	Concevoir une interface de gestion bancaire en architecture microservices : l'interface web n'accède qu'à des API gérant l'authentification, les opérations clients (retrait, dépôt, virement), la validation par un agent et la journalisation des opérations.
Durée et échéances	2 semaines — du 16 au 25 juin 2026. Dépôt du diagramme de Gantt sur Moodle, puis développement, intégration et soutenance bilan.
Technologies utilisées	Python, JWT, bcrypt, API REST, service de messagerie (logs), base de données, Docker Compose, Git.
Compétences développées	▶ Développer — concevoir une architecture microservices et exposer des API sécurisées. ▶ Orchestrer — conteneuriser et déployer les services avec Docker Compose.
Plan d'action	▶ Cadrage et découpage en microservices, mise en place du dépôt Git. ▶ Ma partie : auth-service (login, bcrypt, JWT), middleware de contrôle d'accès par rôles, service de logs, Docker Compose et réseau des conteneurs. ▶ Partie binôme : schéma BDD, account-service, opérations, validation, frontends client et agent. ▶ Intégration, tests bout-en-bout, rapports individuels, présentation bilan.

Journal de bord

Mes traces de travail au fil des deux semaines

16–17 juin · cadrage

TÂCHES RÉALISÉES

Avec Tom, on a commencé par poser à plat les fonctionnalités attendues et découper l'appli en services. On a fait le diagramme de Gantt ensemble et je l'ai déposé sur Moodle, puis on a initialisé le dépôt Git et la structure des dossiers.

DIFFICULTÉS RENCONTRÉES / SOLUTIONS APPORTÉES

Au début je voyais mal où s'arrêtait l'auth et où commençait la gestion des comptes, on a perdu un peu de temps là-dessus. Ce qui nous a débloqués : écrire noir sur blanc le rôle de chaque service avant de toucher au code, comme un mini contrat d'API.

18–19 juin · authentification

TÂCHES RÉALISÉES

J'ai attaqué l'auth-service : la route de login, le hachage des mots de passe avec bcrypt, et la génération des jetons JWT à la connexion.

DIFFICULTÉS RENCONTRÉES / SOLUTIONS APPORTÉES

Je me suis pris la tête sur la vérification du token : au départ je refaisais le contrôle dans chaque route, c'était lourd et ça se répétait partout. J'ai fini par sortir toute la vérif du JWT dans un middleware que les autres services peuvent réutiliser. Beaucoup plus propre.

20–21 juin · contrôle d'accès

TÂCHES RÉALISÉES

J'ai enchaîné sur le middleware de contrôle d'accès par rôle : un client ne doit pas voir ce qu'un agent voit. J'ai limité les accès aux interfaces et aux API selon le profil.

DIFFICULTÉS RENCONTRÉES / SOLUTIONS APPORTÉES

Le piège c'était de bien séparer client et agent sans dupliquer la logique. J'ai mis le rôle directement dans le jeton et je le vérifie au moment des actions sensibles, comme ça la règle est au même endroit.

22–23 juin · logs & conteneurs

TÂCHES RÉALISÉES

J'ai fait le service de logs (chaque API publie ses logs, un service les récupère et les stocke), puis je me suis occupé du Docker Compose et du réseau entre les conteneurs.

DIFFICULTÉS RENCONTRÉES / SOLUTIONS APPORTÉES

Faire se parler les conteneurs m'a demandé pas mal d'essais, au début un service n'arrivait pas à en joindre un autre. En passant par un réseau Docker dédié et en faisant transiter les logs par la messagerie, chaque service est resté indépendant et ça a fini par tourner.

24–25 juin · intégration & bilan

TÂCHES RÉALISÉES

On a tout assemblé avec Tom et testé le parcours complet. J'ai rédigé mon rapport individuel et on a préparé la présentation bilan.

DIFFICULTÉS RENCONTRÉES / SOLUTIONS APPORTÉES

Le truc qui coïncait, c'était de suivre une opération « en attente de validation » qui passe d'un service à l'autre sans se perdre. On l'a déroulé de bout en bout, du client jusqu'à la validation par l'agent, jusqu'à ce que le parcours soit bon.

Fiche synthèse du projet pour le portfolio

Version valorisante du projet — celle qui rejoint le portfolio

Titre de la SAÉ	Application de gestion bancaire en microservices
Contexte et objectif	Concevoir une application bancaire maintenable et évolutive, découpée en services indépendants plutôt qu'en bloc unique, avec authentification, opérations soumises à validation et journalisation centralisée.
Technologies utilisées	Python, JWT, bcrypt, API REST, messagerie (logs), base de données, Docker Compose, Git.
Compétences mobilisées	Développer (architecture microservices, API sécurisées) et Orchestrer (conteneurisation, déploiement Docker Compose).
Déroulement et méthodologie	Projet en binôme sur 2 semaines, planifié au diagramme de Gantt. Répartition claire : authentification / sécurité / infrastructure pour moi, métier / frontend pour mon binôme. Travail incrémental, service par service, puis intégration commune.
Illustrations	Diagramme d'architecture des microservices et diagramme de Gantt (jointes au portfolio).
Retour d'expérience	Ce que je retiens, c'est que le plus dur en microservices ce n'est pas d'écrire le code d'un service, mais de les faire coopérer proprement. Définir le rôle de chaque service avant de coder et découpler la journalisation par la messagerie sont deux réflexes que je garderai pour n'importe quelle architecture distribuée.
Liens	Dépôt GitHub de gestion de projet du groupe · portfolio en ligne.